Agentic Persona Generation in Industry: An In-Situ Evaluation and Lessons Learned (Supplementary Material)

Mohammad Hossein Amini*†, David Dewar†, Shiva Nejati*, Mehrdad Sabetzadeh*
* University of Ottawa, Canada
Email: {mh.amini, snejati, m.sabetzadeh}@uottawa.ca
† Kinaxis, Canada
Email: ddewar@kinaxis.com

I. Prompt Elements used in PerGent

In this section, we present the detailed outlines of prompt elements, described in Section III of the original paper, for each of PerGent's agents. Figures 1, 2 show the detailed prompt outlines for each of the persona generator and critic agents in PerGent. We use different colors to represent each of the Role Definition, Instructions, Action List, Memory, History and Output elements in each agents prompt outline.

II. Prompt Elements used in baselines

Of the baselines, the prompt outlines for each of the persona generator and persona critic agents in PerGentNoRes are identical with the PerGent prompts (see figures 1 and 2), with the external resources removed from their memory. The prompt outline for the OneShot and OneShot+Res are shown in, respectively, figures 3 and 4.

III. LLM-as-Judge Calibration Process

To ensure that the LLM-as-Judge (Gemini 2.0) is an accurate proxy for human experts, we calibrate it through iterative prompt refinement and validation against these experts. We design the prompt for the LLM-as-Judge following the established best practices for LLM prompting, based on the principles of (i) clarity and specific task definition, (ii) role assignment to guide the model's perspective and (iii) contextual grounding with domain-specific information, consistent with OpenAI and Anthropic prompt engineering guidelines [1], [4]. The calibration follows an iterative two-step process: (1) configuring the LLM-as-Judge's prompt and (2) validating its outputs against expert annotations.

In the configuration step, we define the LLM-as-Judge's instructions, including item-persona relationship labels (Table I), example pairs, and the expected format for rationales accompanying each item-persona relationship classification. The initial prompt used in this step is shown in Figure 5. In the validation step, we randomly select 100 item–persona pairs and prompt the LLM-as-Judge to classify their relationships and provide rationales for each. Two independent domain experts, who received the same instructions and examples as the LLM-as-Judge but do not have access to its classifications and rationales, then manually provide relationship labels for the same pairs of persona items. Afterward, they review the LLM-as-Judge's outputs and categorize each rationale as either *justified* or *unjustified*. We evaluate the reliability of the LLM-as-Judge in two ways. First, we measure interrater reliability

```
[ROLE]
You are a persona generator and refiner agent collaborating with a persona critic agent to
build a high-quality persona.

[INSTRUCTIONS]
Generate or refine a professional persona for the given role.

The persona must contain these sections:
- Jobs to be Done
- Pain Points
- Required Skills
- Tools Used

Rules:
- Use information from the external resources whenever possible.
- Ensure items are relevant to the role described.
- Avoid redundancy or vague statements.
- Each section must contain concise items.
- If critic feedback exists, revise the previous persona to address it.

[ACTION LIST]
1. GENERATE_PERSONA
   Create a complete persona from the target persona description and external resources.
2. REFINE_PERSONA
   Improve an existing persona by addressing critic feedback and resolving issues such as
   incompleteness, imprecision, redundancy, or lack of grounding.

Choose the appropriate action based on the conversation history.

[MEMORY]
Target Persona Description:
<PERSONA_DESCRIPTION>

External Resources:
<RESOURCE_1>
<RESOURCE_2>
...
<RESOURCE_N>

[HISTORY]
Previous persona drafts and critic feedback:
<HISTORY_CONTENT>

[OUTPUT]
Return ONLY valid JSON.
{
  "jobs_to_be_done": [
    "<item>",
    "<item>"
  ],
  "pain_points": [
    "<item>",
    "<item>"
  ],
  "required_skills": [
    "<item>",
    "<item>"
  ],
  "tools_used": [
    "<item>",
    "<item>"
  ]
}
```

Fig. 1: Prompt outline for the LLM used in the persona generator agent of PerGent

```
[ROLE]
You are a persona critic agent collaborating with a persona generator and refiner agent to
build a high-quality persona.

[INSTRUCTIONS]
Evaluate the generated persona and determine whether it satisfies the quality requirements.

Assess the persona according to:

- Completeness: all required sections present with sufficient items
- Relevance: content aligns with the target role
- Grounding: information reflects the provided resources when applicable
- Precision: items are clear and specific
- Non-redundancy: avoid duplicated or logically inconsistent items

If problems exist, provide feedback explaining what should be improved.
If the persona is satisfactory, approve it.

[ACTION LIST]
1. CRITIQUE_PERSONA
Identify issues in the persona and produce feedback for refinement.
2. APPROVE_PERSONA
Approve the persona if it satisfies all evaluation criteria.

Choose the appropriate action based on the conversation history.

[MEMORY]
Target Persona Description:
<PERSONA_DESCRIPTION>

External Resources:
<RESOURCE_1>
<RESOURCE_2>
...
<RESOURCE_N>

[HISTORY]
Previous persona drafts and critic feedback:
<HISTORY_CONTENT>

[OUTPUT]
If action = CRITIQUE_PERSONA:
FEEDBACK
- Issue: ...
Section: ...
Recommendation: ...

If action = APPROVE_PERSONA:
APPROVED
```

Fig. 2: Prompt outline for the LLM used in the persona critic agent of PERGENT

```
[ROLE]
Your task is to generate a high-quality persona.

[INSTRUCTIONS]
Generate a professional persona for the given role.

The persona must contain these sections:
- Jobs to be Done
- Pain Points
- Required Skills
- Tools Used

Rules:
- Ensure items are relevant to the role described.
- Avoid redundancy or vague statements.
- Each section must contain concise items.

[MEMORY]
Target Persona Description:
<PERSONA_DESCRIPTION>

[OUTPUT]
Return ONLY valid JSON.
{
  "jobs_to_be_done": [
    "<item>",
    "<item>"
  ],
  "pain_points": [
    "<item>",
    "<item>"
  ],
  "required_skills": [
    "<item>",
    "<item>"
  ],
  "tools_used": [
    "<item>",
    "<item>"
  ]
}
```

Fig. 3: Prompt outline for the LLM used in the ONESHOT

between its relationship labels and those of the domain experts using Krippendorff's α. Second, we report the percentage of its *justified* and *unjustified* rationales as another indicator of alignment between the LLM-as-Judge's reasoning and expert

```
[ROLE]
Your task is to generate a high-quality persona.

[INSTRUCTIONS]
Generate a professional persona for the given role.

The persona must contain these sections:
- Jobs to be Done
- Pain Points
- Required Skills
- Tools Used

Rules:
- Use information from the external resources whenever possible.
- Ensure items are relevant to the role described.
- Avoid redundancy or vague statements.
- Each section must contain concise items.

[MEMORY]
Target Persona Description:
<PERSONA_DESCRIPTION>

External Resources:
<RESOURCE_1>
<RESOURCE_2>
...
<RESOURCE_N>

[OUTPUT]
Return ONLY valid JSON.
{
  "jobs_to_be_done": [
    "<item>",
    "<item>"
  ],
  "pain_points": [
    "<item>",
    "<item>"
  ],
  "required_skills": [
    "<item>",
    "<item>"
  ],
  "tools_used": [
    "<item>",
    "<item>"
  ]
}
```

Fig. 4: Prompt outline for the LLM used in the ONESHOT+RES

TABLE I: Possible relationships between an item p of a persona P and another persona Q, defined by the extent to which the persona-related concepts (jobs to be done, pain points, skills, and tools) mentioned in p are presented in Q.

Relationship	Definition	Example
Fully Present	All persona-related concepts mentioned in p appear in Q, possibly across multiple items or with different wording.	Item p: "Inventory levels are often too low or too high." Persona Q contains: "Products run out unexpectedly" and "Excess items remain in storage."
Partially Present	Only some of the persona-related concepts mentioned in p appear in Q; the rest are missing.	Item p: "Inventory levels are often too low or too high." Persona Q contains: "Products run out unexpectedly."
Not Present	None of the persona-related concepts mentioned in p appear anywhere in Q.	Item p: "Inventory levels are often too low or too high." Persona Q contains no item related to inventory levels.

judgment.

If the LLM-as-Judge does not meet reliability expectations, indicated by a low Krippendorff's α or a low percentage of justified rationales, we refine its prompt using insights from the experts' annotations and repeat the calibration process. Through this iterative refinement we ensure that the LLM aligns closely with human judgment before being used to compute distinctness and preservation on the full set of personas.

LLM-as-Judge calibration results. Table II presents Krippendorff's α between the LLM-as-Judge and domain experts in item-persona relationship classification across two calibration iterations. As shown in the table, in the first iteration of calibration, Krippendorff's α is 0.67 and 0.68 with the two

```
[ROLE]
You are a domain expert evaluating overlap between two personas for computing distinctness and
preservation.

[TASK]
Given (i) one persona item p and (ii) the full text of another persona Q, classify the item--persona
relationship as:
- fully present
- partially present
- not present
and provide a brief rationale.

[DEFINITIONS]
- Persona-related concepts are those about jobs-to-be-done, pain points, skills, and tools.
- Fully present: ALL persona-related concepts in item p appear somewhere in persona Q (possibly spread
across multiple items and/or paraphrased).
- Partially present: SOME (but not all) persona-related concepts in item p appear somewhere in persona Q.
- Not present: NONE of the persona-related concepts in item p appear anywhere in persona Q.

[EXAMPLES]
Example 1 (fully present)
Item p:
"Uses MS Excel for data entry and reporting."
Persona Q (excerpt):
"...spends much of the day working with Excel to enter data and generate reports..."
Output:
Label: fully present
Rationale: Q explicitly mentions spreadsheets for data entry and report generation, which covers the tool
and tasks in p.

Example 2 (partially present)
Item p:
"Builds dashboards in Power BI and writes SQL queries to extract data."
Persona Q (excerpt):
"...creates dashboards in Power BI for stakeholders..."
Output:
Label: partially present
Rationale: Q covers building dashboards in Power BI but does not mention SQL or querying to extract data.

Example 3 (not present)
Item p:
"Automates deployments using GitHub Actions and Docker."
Persona Q (excerpt):
"...works mainly on gathering requirements and coordinating with stakeholders; does not do
implementation..."
Output:
Label: not present
Rationale: Q contains no mention of CI/CD automation, GitHub Actions, Docker, or related deployment
activities.

[INPUT]
Item p:
<<<ITEM_TEXT>>>

Persona Q (all sections/items):
<<<PERSONA_Q_TEXT>>>

[OUTPUT FORMAT] (return exactly two lines)
Label: <fully present | partially present | not present>
Rationale: <1--2 sentences briefly justifying the label by pointing to matched/missing concepts in Q; no extra
text>
```

Fig. 5: Prompt outline for the LLM-as-Judge in its first iteration of calibration

experts, respectively. According to the rationale classification performed by the domain experts, an average of 92% of the rationales provided by the LLM-as-Judge were found to be *justified*.

We manually inspected the cases in which the LLM-as-Judge's classifications were different from those of the domain experts to understand the underlying sources of disagreement. Most mismatches are due to subjective generalization. For example, when comparing a *Pain Points* item, "MS Excel" for data entry with a persona mentioning the same pain point associated more broadly to "spreadsheets", the domain experts considered the relationship *fully present*. In contrast, the LLM-as-Judge judged them *partially present*, arguing that "spreadsheets" could refer to tools other than MS Excel. Notably, the experts agreed that the rationale provided by the LLM-as-Judge was *justified*, even though they still regarded the relationship as *fully present*. Based on these observations, we consulted the domain experts to clarify how such generalization cases should be treated and updated the LLM-as-Judge's prompt accordingly by refining its instructions and illustrative examples.

We then repeated the procedure using a new set of 100 randomly sampled item pairs. Table II presents Krippendorff's α between the LLM-as-Judge and domain experts in item-persona relationship classification during the second iteration of calibration. As shown, Krippendorff's α increased substantially, reaching 0.86 and 0.87 with the two experts, respectively. According to the domain experts' rationale assessments, 96.5% of the LLM-as-Judge's rationales were deemed *justified*. We also measured the interrater agreement between the two experts using Cohen's Kappa (κ), obtaining a value of 0.73, which indicates substantial expert–expert alignment [2]. At this stage, the domain experts considered the LLM-as-Judge annotations to be reliable.

TABLE II: Krippendorff's α and justified rationale rates for item–persona relationship labels produced by the LLM-as-Judge compared to domain experts across two calibration iterations.

Domain Experts	First Iteration of Calibration		Second Iteration of Calibration	
	Krippendorff's α	Justified Rationales	Krippendorff's α	Justified Rationales
Expert 1	0.67	91%	0.86	95%
Expert 2	0.68	93%	0.87	98%

LLM-as-Judge Calibration Summary

Evaluations by two domain experts show that the LLM-as-Judge achieves an average Krippendorff's α of 0.865, indicating strong agreement with human experts [3]. The experts, on average, found 96.5% of its rationales to be *justified* and deemed the LLM-as-Judge reliable.

References

[1] anthropics. Anthropic's prompt engineering interactive tutorial. https://github.com/anthropics/prompt-eng-interactive-tutorial, 2024. GitHub repository, accessed 2026-02-10.

[2] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[3] Klaus Krippendorff. *Content Analysis: An Introduction to Its Methodology*. Sage Publications, Thousand Oaks, CA, 4 edition, 2018.

[4] OpenAI. Prompt engineering guide. https://developers.openai.com/api/docs/guides/prompt-engineering, 2026. OpenAI API documentation, accessed 2026-02-10.